

Esame OS Internals (Cabodi)

Data: 08/07/2022

Nota Generale: TUTTE LE RISPOSTE SI / NO DEVONO ESSERE MOTIVATE. QUANDO I RISULTATI SONO NUMERI, SONO RICHIESTI SIA IL RISULTATO FINALE SIA I RELATIVI PASSI INTERMEDI (O FORMULE).

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale e Algoritmo PFF)

Contesto: Si consideri, per un dato processo, la seguente stringa di accessi alla memoria. Per ogni indirizzo (indirizzamento Byte, con indirizzi espressi in codice esadecimale) viene riportata anche l'operazione di lettura (R)/ scrittura (W):

R 33FB, R 2264, W 0AD3, W 237E, R 08C8, W 13D7, R 094A, R 3465, W 32A0, R 1BB0, W 0CE5, R 1480, R 3294, R 2AB8.

Supponiamo che gli indirizzi fisici e logici siano rappresentati su 16 bit, la dimensione della pagina sia 2 KByte e 5D02 sia l'indirizzo massimo utilizzabile dal programma (il limite superiore dello spazio degli indirizzi).

Si risponda alle seguenti domande:

- A) Calcolare la dimensione dello spazio di indirizzamento (espressa come numero di pagine) e la frammentazione interna.
- A) Calcolare la stringa degli accessi alle pagine.
- A) Simulare un algoritmo di sostituzione delle pagine di tipo PFF (Page Fault Frequency), assumendo un'unica soglia di tempo $C = 2$. Rappresentare il resident set (frame fisici contenenti pagine logiche) dopo ogni accesso alla memoria. Indicare esplicitamente le attivazioni dell'algoritmo (la procedura di selezione della vittima). Rappresentare anche (con un pedice o altra notazione) i bit di riferimento associati a pagine / frame.

L'algoritmo PFF (approssimato) può essere riassunto come segue: L'algoritmo viene attivato ad ogni page fault, non ad ogni accesso a pagina. A seconda dell'intervallo di tempo TAU dal page fault precedente:

- se $TAU < C$ (frequenza di PF maggiore di quella desiderata), aggiungere un nuovo frame al Resident Set del processo.
- se $TAU \geq C$ (frequenza di PF OK), rimuovere dal resident set tutte le pagine con bit di riferimento a 0; quindi azzerare il bit di riferimento di tutte le altre pagine nel Resident Set.

NOTA: Quando si valuta TAU , considerare solo gli istanti di tempo "tra" i due page fault (es. PF ai tempi 7 e 8 $\rightarrow TAU = 0$; PF ai tempi 12 e 15 $\rightarrow TAU = 2$). I bit di riferimento vengono posti a 1 a ciascun accesso e azzerati come ultimo passaggio dell'algoritmo. Ogni volta che i bit vengono azzerati, si include la pagina relativa al PF corrente.

- A) Mostrare i page fault (accessi a pagine esterne al resident set) e calcolare il loro conteggio complessivo.
-

Domanda 2 (RAID e Affidabilità)

Contesto: Sia dato un disco organizzato con struttura RAID. Sono dati il tempo medio tra guasti (MTTF: Mean Time To Failure), il tempo medio per riparazione (MTTR: Mean Time To Repair), e il tempo medio tra perdite di dato (MTTDL: Mean Time To Data Loss).

Si risponda alle seguenti domande:

- A) A causa di vincoli aggiuntivi, la probabilità di guasto di un disco, mentre l'altro è in riparazione, viene RADDOPPIATA. Si può considerare che questo sia equivalente a:
- A1) moltiplicare MTTR per 2?
 - A2) moltiplicare MTTR per 0.5?
 - A3) Lasciare tutto com'è (senza modifiche) in quanto MTTR è trascurabile?
- A) Si consideri una struttura RAID con 2 dischi in configurazione "mirrored". Se ognuno dei dischi può guastarsi in modo indipendente dall'altro, con $MTTR = 36$ ore, quanto deve essere MTTF per garantire $MTTDL > 100 \cdot MTTF$?
-

Domanda 3 (Ottimizzazioni Memoria Virtuale)

Contesto: Si considerino le ottimizzazioni implementate per migliorare le prestazioni complessive della gestione della memoria virtuale.

Si risponda alle seguenti domande:

- A) Spiegare brevemente la tecnica COW (Copy on Write): che cosa è e come migliora le prestazioni della gestione della memoria? I miglioramenti sono in termini di tempo? Di occupazione di memoria? Di entrambi tempo e memoria? (Motivare).
- A) Spiegare perché l'I/O sulla partizione di swap (backing store) è più veloce dell'I/O mediante file system (anche se a parità di disco). E' possibile che lo spazio di indirizzamento virtuale di un processo sia più grande del numero di blocchi totali disponibili su backing store? (Motivare).
- A) Quale è il ruolo del bit di validità (valid/invalid bit) associato a una pagina? Il validity bit viene memorizzato solo nella tabella delle pagine oppure viene anche immagazzinato nella TLB (quando disponibile su un dato processore)? (Motivare).

2 SECTION 2: OS161

Domanda 4 (Gestione Console e Sincronizzazione)

Contesto: Si consideri la gestione della console in OS161 e le funzioni scritte di seguito.

```
void putch_intr (struct con_softc *cs, int ch) {
    P(cs->cs_wsem);
    cs->cs_send(cs->cs_devdata, ch);
}
```

```
void con_start(void *vcs) {
    struct con_softc *cs = vcs;
    V(cs->cs_wsem);
}
```

Si risponda alle seguenti domande:

- B) Siccome `putch_intr` è chiamata da `putch(int ch)`, e si tratta di un output (del carattere `ch`), perché `putch_intr` aspetta (chiamando P) invece di segnalare (V)?
 - B) Fornire la definizione del campo `cs_wsem` nella `struct con_softc`.
 - B) Fornire un'implementazione delle due funzioni, sostituendo il semaforo `cs_wsem` con una condition variable (definire anche i campi necessari in `struct con_softc`).
-

Domanda 5 (Strutture Processi e Thread)

Contesto: Sia dato un sistema OS161, nella versione base. Si considerino le definizioni delle `struct thread` e `struct proc` parzialmente qui riportate:

```
struct thread {
    char *t_name;
    void *t_stack;
    struct switchframe *t_context;
    struct cpu *t_cpu;
    struct proc *t_proc;
    /* add more here as needed */
};
```

```
struct proc {
    unsigned p_numthreads;
    struct addrspace *p_addrspace;
    struct vnode *p_cwd;
    /* add more here as needed */
};
```

Si risponda alle domande che seguono:

- B) Si dica come sono correlate/collegate le due struct nel caso in cui un processo abbia più di un thread.

- B)** Si dica anche se, dato un puntatore `p` a una `struct proc`, sia possibile ricavare l'elenco di tutti i thread del processo e stamparne il nome. Qualora non sia possibile, si proponga come modificare (aggiungere campi) a `struct proc` e/o `struct thread`, in modo da poter realizzare l'operazione (dato un processo, elencare i suoi thread). Scrivere la funzione `printThreadNames(struct proc *p)` che svolge tale compito.
- B)** Si spieghi brevemente il ruolo del puntatore `t_context` nella `struct thread`.